



Mobile Robot Vision Expert

M4-G

GMSL Integration

and

SDK Developer Guide

Based on OpenCL/CUDA

Item	Details
Document version	V1.1
Applicable product	M4-G RGBD camera
Interface	GMSL2
SDK dependency	OpenCL/CUDA

Contents

1 Overview	1
1.1 Purpose	1
1.2 Scope	1
1.3 System Architecture	2
1.4 Integration Workflow	2
2 Support Scope	2
2.1 Deliverables and Responsibilities	2
2.2 Limitations	3
3 OpenCL/CUDA Environment and Version Management	4
3.1 Environment Requirements	4
3.2 Version and Dependency Management	4
4 GMSL Integration	5
4.1 Hardware Parameters	5
4.2 Data Channels	6
4.3 SerDes Initialization	6
4.4 Triggering and Stream Control	7
4.5 Temporary Trigger Program	7
4.6 Multiple Cameras	7
5 Linux Driver Porting and Raw Data Capture	8
5.1 Reference Driver	8
5.2 Driver Porting Tasks	9
5.3 Driver and Link Validation	9
5.4 Raw Data Capture with v4l2-ctl	9
5.5 Using capture.c	10
5.6 Mapping Raw Frames to DMSDK	10
6 DMSDK Integration and Data Processing	11
6.1 SDK Components	11
6.2 Build Environment	12

6.3 Build and Deployment	12
6.4 Checking SDK Libraries	12
6.5 OpenCL and CUDA Runtime Configuration	14
6.6 Core APIs	14
6.7 Minimal API Workflow	15
6.8 Alignment Modes and Output Formats	16
6.9 Example Program Scope	16
6.10 Common Error Codes	17
7 OTA Updates	18
7.1 Tool Options	18
7.2 Pre-update Checks	18
7.3 Common Commands	19
7.4 Platform-specific Limitations	19
8 Troubleshooting	19

1 Overview

1.1 Purpose

This guide describes GMSL2 hardware integration, SerDes configuration responsibilities, driver porting, raw data capture, DMSDK integration, and OTA updates for the M4-G RGBD camera with the OpenCL/CUDA SDK.

First, integrate the deserializer, MIPI CSI, Camera Framework/V4L2, and trigger path on the target platform. Then use DMSDK to process raw TOF and RGB data into depth images, RGB images, aligned RGBD images, and point clouds.

1.2 Scope

Item	Description
Product model	M4-G RGBD camera
Interface type	GMSL2; MAX96717 serializer built inside the camera
Host platform	Linux aarch64, Linux x86_64, or Android aarch64, with the corresponding OpenCL or CUDA runtime environment
Programming interface	C/C++; examples use C++14, V4L2, and the Linux I2C interface
SDK backend	OpenCL/CUDA
Output data	Depth images, RGB images, aligned RGBD images, and point clouds
Driver scope	The mrdvs.c reference camera sub-device driver and interface parameters are provided. The customer is responsible for the deserializer driver, device tree, Camera Framework, triggering, and address mapping.
Example programs	capture.c for streaming, trig.c for temporary triggering, and processing examples such as demo_module and demo_session_module in the DMSDK package

1.3 System Architecture

Camera	Transport link	Host reception	Data processing	Customer application
M4-G MAX96717	GMSL2 coaxial cable	Deserializer → MIPI CSI → Camera Framework/V4L2	DMSDK OpenCL/CUDA backend	Depth, RGB, aligned images, and point clouds

1.4 Integration Workflow

1. Identify the target SoC, carrier board, BSP, Linux kernel, deserializer model, camera count, and Camera Framework. Select the OpenCL or CUDA runtime environment.
2. Use a reference deserializer board for a quick evaluation, or connect the MAX96717 to the target deserializer on the customer's own board.
3. Implement the deserializer driver and configure I2C addresses, SerDes initialization, Frame Sync triggering, CSI-2 VC/Data Type, and the device tree.
4. Use mrdvs.c as a reference to port the camera sub-device driver and media bus parameters, and expose video nodes for VC0 and VC1.
5. Use v4l2-ctl or capture.c to verify the raw TOF stream on VC0 and the raw RGB stream on VC1.
6. Deploy the OpenCL or CUDA version of DMSDK for the target platform. Implement device opening, raw data processing, RGBD alignment, point cloud output, and resource cleanup.
7. Validate link stability, synchronization, processing performance, memory stability, and OTA updates for the single-camera or dual-camera configuration.

2 Support Scope

2.1 Deliverables and Responsibilities

Category	Provided by MRDVS	Customer responsibility	Joint validation
Hardware integration	M4-G interface parameters, reference SerDes configuration, and a reference deserializer board available on request for the project	Host carrier board, deserializer hardware, power supply, cabling, EMC, and mechanical integration	Link lock, I2C access, and CSI output

Category	Provided by MRDVS	Customer responsibility	Joint validation
SerDes configuration	Reference MAX96724 – MAX96717 register settings and initSerDer.txt	Deserializer driver, address mapping, link ports, VC/pipe mapping, Frame Sync, startup sequence, and production integration	VC/Data Type, lanes, data rate, triggering, and synchronization
Linux driver	mrdvs.c reference V4L2 camera sub-device driver and image parameters	Porting to the target BSP kernel, device tree, deserializer driver, Camera Framework, and build integration	Node enumeration, format negotiation, and continuous streaming
SDK	DMSDK libraries, headers, examples, release notes, and API support	Runtime dependencies, application code, threading model, memory management, and deployment	Output correctness, performance, and stability
OTA	OTA tool, instructions, and firmware files	Target-platform GPIO and power control, service management, and update workflow integration	Version queries, updates, and recovery after power loss

2.2 Limitations

- This guide covers both the OpenCL and CUDA SDK variants. They expose the same API. Deploy the matching shared libraries for the target platform and selected backend.
- GMSL/CSI transports raw camera data to the host; OpenCL/CUDA performs algorithm processing on the host. These operate at different layers and cannot replace one another. The target platform requires a working data link, GPU drivers, and a compatible system ABI.
- The current SDK and examples support up to two M4-G cameras. Address management, capture, algorithm instances, and resource scheduling for more than two cameras are not supported.
- mrdvs.c uses Rockchip 5.10 interfaces and is provided as a reference only. It is not a production-ready replacement for drivers on other platforms or kernel versions.
- The example I2C buses, device addresses, video nodes, and register values may differ from those on the customer's platform. Check them before use.

3 OpenCL/CUDA Environment and Version Management

3.1 Environment Requirements

Category	Supported configuration
Hardware platform	The package provides libraries for Linux aarch64, Linux x86_64, and Android aarch64. Support depends on validation of the BSP, drivers, and system environment on each platform.
Operating system	Linux; no specific Ubuntu distribution or release is required.
Kernel/BSP	Use a Linux kernel and BSP compatible with the target hardware and the OpenCL or CUDA drivers.
Build environment	The compiler toolchain and system runtime libraries must be compatible with the SDK release package.
GPU runtime environment	OpenCL requires a GPU driver, an OpenCL ICD Loader, and the corresponding runtime. CUDA requires an NVIDIA driver and a CUDA runtime environment compatible with the SDK shared libraries. Validate the specific versions on the target platform.

Select the platform directory as follows:

Platform directory	OS/CPU architecture	GPU backend	Target platform
linux_aarch64	Linux aarch64	OpenCL	General ARM64 platforms, typically RK3588
linux_aarch64_3576	Linux aarch64	OpenCL	RK3576
linux_x64	Linux x86_64	OpenCL	Linux x86_64
android_aarch64	Android aarch64	OpenCL	Android ARM64
cuda	Linux aarch64	CUDA	NVIDIA ARM64

3.2 Version and Dependency Management

- Deploy libdmsdk.so and libdmprocess.so together from the same platform directory in the same release package. Do not replace either library independently or mix versions or backends.

- Archive the headers, main library, algorithm library, and release_note.txt together as a complete release.
- For operation and troubleshooting, record the SDK package identifier, the value returned by DmGetApiVersion, the algorithm library version, the selected platform directory, the system image, and the OpenCL or CUDA runtime environment version.
- After an SDK update, revalidate raw data packing, RGB output format, alignment modes, point clouds, processing performance, dual-camera synchronization, and memory stability.

4 GMSL Integration

4.1 Hardware Parameters

Item	Parameter	Description
Serializer	MAX96717; 7-bit address 0x40	Built into M4-G; GMSL2 at 6 Gbps; Pixel Mode
FPGA	7-bit address 0x11	Register 0x2000: 0 = stream off; 1 = stream on
Reference deserializer	MAX96724; example address 0x27	Reference project configuration; the actual address depends on the customer's hardware
MIPI CSI-2	4 Lane	The lane rate depends on the supplied SerDes configuration and target-platform validation. The reference script sets it to 1.5 Gbps.
Frame Sync	MAX96717 MFP0	The customer configures the deserializer Frame Sync output. trig.c is for temporary testing only.
Reference link	MAX96724-MAX96717	initSerDer.txt defaults to one camera and does not include I2C address mapping.

***Note:** The M4-G has a built-in MAX96717 serializer. When using a MAX96712, MAX96724, or another deserializer on the host side, configure the registers, drivers, CSI topology, address mapping, and triggering according to the device documentation and the actual carrier board.

4.2 Data Channels

Channel	Data type	Resolution	CSI-2 data format	Reference host format
VC0	Raw TOF 3D image	2560×960	RAW8,0x2A	GREY (default V4L2 FourCC). Some platforms or drivers may enumerate it as RGGB/SBGGR8. In all cases, treat the data as a raw byte stream for the TOF algorithm.
VC1	RGB 2D image	1280×1056	YUV422 8bit,0x1E	UYVY

The effective depth resolution is 640×480 . The 2560×960 frame on VC0 is a raw TOF transport frame, not a directly displayable depth image. Interpret it using the DMSDK input structure and process it with the algorithm library.

4.3 SerDes Initialization

initSerDer.txt is a reference register script for a single-camera MAX96724–MAX96717 link. The defaults are I2C bus 4, deserializer address 0x27, serializer address 0x40, and M4-G FPGA address 0x11.

The script illustrates the register configuration sequence and supports laboratory validation only. Adapt it to the actual deserializer, I2C bus, link port, address plan, CSI port, and BSP requirements. Integrate it into the customer's deserializer driver or system initialization sequence.

Variable	Default	Meaning
BUS	4	I2C bus number
DES	0x27	MAX96724 deserializer address
SER	0x40	MAX96717 serializer address
CAM	0x11	M4-G FPGA 7-bit address

1. Stop camera streaming.
2. Enable the target GMSL link and control channel, and wait for the link to stabilize.
3. Configure the MAX96717 MIPI input, video transmission, and VC/Data Type mapping.
4. Configure the MAX96724 video pipes, MIPI PHY, lane mapping, CSI rate, and output port.
5. Configure MFP/GPIO and the Frame Sync path.

6. Enable serializer video transmission, enable the deserializer CSI output, and perform a link reset.
7. Check GMSL lock, I2C access, and CSI reception before starting the stream.

4.4 Triggering and Stream Control

For M4-G to output images, the SerDes link must be operational, host CSI reception must be running, a valid trigger signal must be present, and the FPGA stream-control register must be set to 1. Configure the trigger source, frequency, pulse width, synchronization accuracy, and startup sequence for the deserializer and system architecture. The recommended sequence is:

1. Initialize the deserializer and serializer, and confirm link lock.
2. Configure and start host MIPI CSI and Camera Framework/V4L2 reception.
3. Start hardware triggering through the deserializer Frame Sync output.
4. Write 1 to M4-G FPGA register 0x2000 to start streaming.
5. Confirm stable frames on both VC0 and VC1. On shutdown, write 0 to stop camera streaming before stopping host reception and triggering.

```
# Stream on
i2ctransfer -f -y 4 w3@0x11 0x20 0x00 1
# Stream off
i2ctransfer -f -y 4 w3@0x11 0x20 0x00 0
```

Some platforms require V4L2 STREAMON before the camera stream-on command; otherwise, the CSI receiver cannot establish a stable data path. Validate the sequence with the target BSP.

4.5 Temporary Trigger Program

trig.c accesses address 0x40 through /dev/i2c-4 and repeatedly writes MAX96717 registers 0x02BF and 0x02BE to generate a simulated trigger on MFP0. It has no external library dependencies and can be used for single-camera laboratory debugging. It is not recommended for synchronization in production.

```
gcc trig.c -o trig
sudo ./trig
```

***Before use:** Adjust the I2C device node, serializer address, trigger period, and pulse width for the target platform. For production, use the Frame Sync configuration in the customer's deserializer driver.

4.6 Multiple Cameras

***Camera limit:** The current SDK supports one or two M4-G cameras, with a maximum of two. Configurations with more than two cameras are not supported.

- `initSerDer.txt` defaults to one camera and does not enable I2C address mapping. It cannot be used unchanged for two cameras.
- For two cameras, plan the deserializer links, serializer aliases, FPGA aliases, VC/pipe mapping, and a shared Frame Sync signal.
- The host must provide four stable video inputs for two cameras: two TOF streams and two RGB streams.
- The application must preserve the TOF/RGB pairing for each camera and verify synchronization using frame numbers, timestamps, or hardware synchronization signals.
- DMSDK examples load buses, addresses, and video nodes from command-line arguments or `.module_type`. A multi-camera application must manage separate handles and frame pairing according to the actual address mapping, media topology, and camera count.

5 Linux Driver Porting and Raw Data Capture

5.1 Reference Driver

`mrdvs.c` is a reference V4L2 sub-device driver based on Rockchip 5.10 interfaces. It illustrates the M4-G media bus formats, resolutions, VC information, frame rate, and video node exposure.

***Not included:** `mrdvs.c` does not implement deserializer register initialization, GMSL link management, I2C address mapping, Frame Sync trigger generation, FPGA stream-control commands, or a complete device tree. It is not a complete GMSL driver ready to load on the target platform.

Item	Reference value
Driver name	MRDVS
DRIVER_VERSION	KERNEL_VERSION(0,0x01,0x13)
XVCLK	24MHz
Link Frequency	750 MHz, corresponding to a DDR lane rate of 1.5 Gbps
Number of lanes	4
Bits per sample	8
VC0 format	MEDIA_BUS_FMT_SBGGR8_1X8, 2560 × 960. VC0 uses GREY FourCC by default at the video node. Use the format enumerated by the target-platform driver.
VC1 format	MEDIA_BUS_FMT_UYVY8_2X8, 1280×1056, Data Type 0x1E

Item	Reference value
Frame rate	30 fps in the reference configuration

5.2 Driver Porting Tasks

- Rewrite or port the sub-device registration, pad, format, frame interval, and channel information interfaces for the target kernel and Camera Framework.
- Implement link initialization, lock detection, I2C aliases, video pipes, CSI output, and Frame Sync in the customer's deserializer driver.
- Configure the I2C nodes, endpoints, lanes, link frequency, clocks, reset, power, and CSI topology in the device tree.
- Ensure that VC0 and VC1 map to the correct CSI receiver entities and video nodes.
- Implement the correct sequence for STREAMON, triggering, camera stream-on, and STREAMOFF in the driver or a system service.
- For two cameras, provide two sets of address mappings and four video nodes. More than two cameras are not supported.

5.3 Driver and Link Validation

After loading the driver, verify that video nodes can be enumerated, formats can be set, and streams can be started and stopped. Common diagnostic commands are shown below.

```
i2cdetect -y 4
media-ctl -p
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video11 --all
v4l2-ctl -d /dev/video12 --all
```

- Verify I2C access to the deserializer, serializer aliases, and M4-G FPGA addresses.
- Check that VC0 and VC1 map to the expected CSI receiver entities and video nodes in the media topology.
- Confirm that the node format, width, height, bytesperline, and sizeimage match the actual data transmission.
- After streaming starts, confirm that frame counts increase continuously, with no CSI CRC/ECC errors, short or long frames, or FIFO overflows.

5.4 Raw Data Capture with v4l2-ctl

```
# VC0: raw TOF stream
v4l2-ctl -d /dev/video11 --set-fmt-
video=width=2560,height=960,pixelformat=GREY \
```

```
--stream-mmap=3 --stream-count=1 --stream-to=mpTOF.bin --stream-poll
# VC1: raw RGB stream
v4l2-ctl -d /dev/video12 --set-fmt-
video=width=1280,height=1056,pixelformat=UYVY \
--stream-mmap=3 --stream-count=1 --stream-to=mpRGB.bin --stream-poll
```

Video node numbers and FourCC values depend on the target-platform driver. Although TOF data may be configured as RGGB/SBGGR8, it is raw algorithm input and must not be interpreted as a conventional Bayer image.

5.5 Using capture.c

capture.c demonstrates V4L2 MMAP streaming, including device capability queries, format configuration, buffer allocation, STREAMON, DQBUF, QBUF, and STREAMOFF. Its default parameters are not M4-G settings and must be changed before use.

***Code changes:** Make the device node, width, height, pixel format, and data-saving behavior configurable at runtime. Validate the data using bytesused, bytesperline, and sizeimage returned by V4L2.

```
gcc capture.c -o capture
sudo ./capture
```

Data	V4L2 width × height	Reference FourCC	Frame size check
Raw TOF data	2560×960	GREY; may appear as RGGB/SBGGR8 on some platforms	Typically 2560 × 960 bytes
Raw RGB data	1280×1056	UYVY	Typically 1280 × 1056 × 2 bytes

5.6 Mapping Raw Frames to DMSDK

Production code must use the actual V4L2 buffer values and DMSDK Imageframe fields. Do not calculate lengths solely from constants in the examples.

Input	V4L2 transport	DMSDK Imageframe fields	Description
TOF	2560×960 RAW8	width=640, height=960, channels=2, type=UNSIGNED_SHORT, size_per_pixel=2	The byte count is 640 × 960 × 2 × 2. Interpret the raw packing according to the SDK format.

Input	V4L2 transport	DMSDK Imageframe fields	Description
RGB	1280×1056 UYVY	width=1280, height=1056, channels=2, type=UNSIGNED_CHAR, size_per_pixel=1	YUV422 input, 2 bytes per pixel

6 DMSDK Integration and Data Processing

6.1 SDK Components

DMSDK processes raw images from M4-G and outputs depth images, RGB images, aligned RGBD images, and point clouds.

Category	File	Purpose
Main library	libdmsdk.so	Device access, parameter management, processing workflow, and public APIs
Algorithm library	libdmprocess.so	Depth computation, RGBD alignment, and OpenCL/CUDA processing
Headers	lx_module_api.h	DMSDK public API declarations
Headers	lx_module_define.h	DMSDK data structures, error codes, and feature enumerations
Headers	lx_camera_define.h, lx_module_version.h	Common camera definitions and DMSDK version macros
Driver source	driver/v4l2, driver/session	Source code for V4L2 data sources and CaptureSession wrappers
Examples	examples	Direct API, Session, and IMU examples, with CMake build files
Release information	release_note.txt	SDK change history, API workflows, and configuration instructions

6.2 Build Environment

Item	Requirement
Architecture	Linux aarch64 (OpenCL/CUDA); the package also includes libraries for Linux x86_64, RK3576, and Android aarch64
Language and build tools	C++14; examples require CMake 3.10 or later
Core dependencies	pthread, dl, and the standard C/C++ runtime libraries
Capture dependencies	Linux V4L2, i2c-dev, mmap/ioctl
GPU backend dependencies	OpenCL: GPU driver, ICD Loader, and runtime; CUDA: NVIDIA driver and a CUDA runtime environment compatible with the shared libraries
Permissions	Access to /dev/video*, /dev/i2c-*, and related device nodes

6.3 Build and Deployment

examples/CMakeLists.txt loads DMSDK from /opt/MRDVS/include and /opt/MRDVS/lib. libdmsdk.so loads libdmprocess.so at runtime. On ARM64 devices, install.sh always installs the OpenCL libraries from SDK/lib/linux_aarch64; it does not detect NVIDIA GPUs or select the cuda directory automatically. For CUDA, manually deploy the matching libraries from SDK/lib/cuda.

```
cd <DMSDK_PACKAGE_DIR>
# OpenCL deployment on general aarch64 platforms
sudo ./install.sh
# CUDA aarch64 deployment (install.sh does not select cuda automatically)
sudo install -d /opt/MRDVS/include /opt/MRDVS/lib
sudo cp -a SDK/include/. /opt/MRDVS/include/
sudo cp -a SDK/lib/cuda/. /opt/MRDVS/lib/
cd examples && mkdir -p build && cd build && cmake .. && make -j
```

6.4 Checking SDK Libraries

The commands in this section are for environment checks and troubleshooting after SDK deployment; they are not application source code. On the target Linux device, open the selected platform directory and check the file architecture, version API, and standard shared-library dependencies. ldd alone cannot determine whether the OpenCL/CUDA backend is available.

```
cd <DMSDK_PACKAGE_DIR>/SDK/lib/<PLATFORM_DIR>
# 1. Check both libraries' file types and target architectures
file ./libdmsdk.so ./libdmprocess.so
# 2. Check that the main DMSDK library exports the version API
nm -D ./libdmsdk.so | grep DmGetApiVersion
# 3. Check standard shared-library dependencies for "not found"
ldd ./libdmsdk.so
ldd ./libdmprocess.so
# 4. Check the GPU backend as described in 6.5; ldd alone is insufficient
```

- **file:** Checks the bitness and target architecture of both .so files. The linux_aarch64, linux_aarch64_3576, android_aarch64, and cuda directories contain 64-bit ARM aarch64 libraries; linux_x64 contains 64-bit x86-64 libraries. If the architecture does not match the target platform, select the correct library directory.
- **nm:** Checks that the main library exports DmGetApiVersion. At runtime, call DmGetApiVersion() to obtain the actual API version and compare it with lx_module_version.h and release_note.txt.
- **ldd:** Checks standard shared-library dependencies. The output must not contain "not found". The algorithm library may load OpenCL/CUDA libraries dynamically, so ldd alone cannot establish GPU backend availability. Also perform the runtime environment checks in Section 6.5 and run the examples.

***Execution environment:** Run these checks on a target device whose architecture matches the selected library directory. Running ldd on aarch64 shared libraries directly on an x86 computer may produce an architecture-mismatch message such as "not a dynamic executable". This does not indicate that the library is corrupt.

Check	Expected result	Corrective action
File architecture	Both libraries match the architecture of the selected platform directory.	Use the SDK libraries for the target platform and selected OpenCL/CUDA backend.
Version information	The main library, algorithm library, headers, and release_note.txt belong to the same release.	Redeploy the complete SDK package. Do not mix versions.
Shared-library dependencies	The ldd output contains no "not found" entries.	Install the missing dependencies and check the runtime library search path.

After these checks pass, run `demo_module` or `demo_session_module`. Resolve library architecture, version, and dependency issues before investigating camera capture or algorithm processing.

6.5 OpenCL and CUDA Runtime Configuration

1. The deployed SDK/lib platform directory determines the OpenCL/CUDA backend. Applications use the same API, and the backend does not switch automatically at runtime.
2. For OpenCL, install the appropriate GPU driver, OpenCL ICD Loader, and runtime.
3. For CUDA, provide an NVIDIA driver and CUDA runtime environment compatible with the libraries, and ensure that `libcuda.so.1` can be loaded.
4. For OpenCL, run `clinfo` and confirm that platforms and devices are enumerated. For CUDA, use `ldconfig` to check that `libcuda.so.1` can be resolved. After the system library checks pass, run an example to confirm that the algorithm uses the GPU correctly.
5. `DmSetGpuEnable(true)` enables the GPU path in the deployed algorithm library; it does not select OpenCL or CUDA. Call it before `DmOpenDevice`. If the GPU backend fails, disable GPU processing and compare with the CPU path to help isolate the problem.

```
clinfo
ls -l /etc/OpenCL/vendors/
ldconfig -p | grep libcuda.so.1
```

After changing the OpenCL/CUDA libraries, revalidate depth, RGB, RGBD alignment, point clouds, processing latency, and memory stability during extended operation.

6.6 Core APIs

API	Purpose
<code>DmGetApiVersion</code>	Get the API version.
<code>DmSetInfoOutput</code>	Configure the log level, console output, and log directory.
<code>DmOpenDevice</code>	Open a device with <code>LxOpenDeviceParams</code> and obtain its handle and device information.
<code>DmOpenDeviceAuto</code>	Legacy device-opening API, marked for deprecation.
<code>DmCloseDevice</code>	Release the device handle and associated resources.
<code>DmStartStream</code>	Start image streaming.
<code>DmStopStream</code>	Stop image streaming.

API	Purpose
DmDemuxFrame	Preprocess raw frames using channel data and the subframe index.
DmImageProcess	Process preprocessed depth, RGB, and alignment data.
DmDepth2PointCloud	Output point cloud data in the specified format.
DmSetIntValue/DmGetIntValue	Set or get integer parameters.
DmSetFloatValue/DmGetFloatValue	Set or get floating-point parameters.
DmGetPtrValue/DmSetPtrValue	Get or set pointer parameters.
DmSetCmd	Execute command operations.
DmUpgrade	Update firmware.
DmCollectImuData	Read IMU data in a blocking call; use a separate thread.
DmSetGpuEnable	Enable or disable GPU acceleration in the algorithm library. Call before opening the device.
DmSetParallelThread	Set the number of parallel data-processing threads.

DmPreProcessData and DmProcessData are newer APIs based on BaseImageFrame and AllImageFrame. They are still being debugged and refined and are intended to replace DmDemuxFrame and DmImageProcess. For production, use one consistent workflow supported by the selected SDK version; do not mix the two sets of data structures.

6.7 Minimal API Workflow

1. Call DmSetInfoOutput to configure the log level, console output, and log directory.
2. Before opening the device, call DmSetGpuEnable(true) and, if needed, DmSetParallelThread to set the number of parallel threads.
3. Populate LxOpenDeviceParams and call DmOpenDevice to obtain DmHandle and LxModuleInfo.
4. Configure parameters with DmSetIntValue, DmSetFloatValue, or DmSetPtrValue, and check the return values.
5. Call DmStartStream to start the device stream.
6. Acquire raw frames from the 3D and 2D data sources for the same camera and capture time. Populate LxChannelData and call DmDemuxFrame.

7. Call `DmImageProcess`, then read depth, RGB, and intensity data from the returned `DemuxData`.
8. Call `DmDepth2PointCloud` when point cloud output is required.
9. Promptly return V4L2 buffers with QBUF after processing.
10. On shutdown, call `DmStopStream` followed by `DmCloseDevice`. Ensure cleanup also covers all error paths.

6.8 Alignment Modes and Output Formats

The RGB output format depends on the SDK version and may differ from the UYVY input format. Use the `Imageframe` fields returned by `DmImageProcess` and `release_note.txt` to determine the width, height, channel count, data type, stride, and buffer length.

Setting	Output	Resolution
<code>RGBD_ALIGN_OFF</code>	RGB and depth remain in their own coordinate systems.	Depth is typically 640×480 ; RGB is 1280×1056 .
<code>DEPTH_TO_RGB</code>	Depth is aligned to RGB.	Depth output matches the RGB dimensions: 1280×1056 in the example.

6.9 Example Program Scope

***Scope:** DMSDK examples do not implement deserializer initialization, address mapping, triggering, or video node creation. Adjust the `initSerDer.txt` path, buses, addresses, and video nodes for the customer's platform.

Check	Example/default	Usage requirements
Direct API examples	<code>demo_module</code> , <code>demo_module2</code>	Demonstrate V4L2 capture, preprocessing, image processing, and point cloud output. Production applications require complete state and error handling.
Session wrapper	<code>demo_session_module</code>	Uses <code>LxCaptureSession</code> to manage opening, capture, processing, stopping, and cleanup.
IMU example	<code>demo_imu</code>	Check <code>bsupport_imu</code> before enabling the IMU, then call <code>DmCollecImuData</code> in a separate thread.

Check	Example/default	Usage requirements
Default control mode	LX_CONTROL_ALL	Reads configuration from the device. Use HALF/NONE as needed for debugging or when the application manages communication.
Configuration source	.module_type or command-line arguments	Check control_type, calib_path, custom_path, buses, addresses, and 3D/2D nodes.
GPU switch	DmSetGpuEnable(true)	Must match the selected OpenCL/CUDA algorithm library and runtime environment.
SerDes script	/userdata/default/initSerDer.txt	Example path only. Adjust for the location, permissions, and startup sequence on the customer's system.
Resource cleanup	stop/release, DmStopStream/DmCloseDevice	Production applications must handle normal shutdown, timeouts, and error paths.

6.10 Common Error Codes

Error code	Meaning	Checks
0	Success	—
-2	Feature not supported	SDK version, device model, and parameter
-3	Handle not initialized or communication error	OpenDevice result, I2C, and handle lifetime
-4	Invalid argument	Null pointers, enumeration values, and value ranges
-11	Image dimensions do not match	width, height, channels, type, size_per_pixel, bytesused
-13	Device connection failed	GMSL, I2C, power supply, and addresses
-14	Device initialization failed	Calibration, firmware, resource paths, and communication

Error code	Meaning	Checks
-15	Target device not found	Device type, grouping, and address mapping
-16	Invalid file	File name, format, permissions, and path
-25	Algorithm library failed to load	libdmprocess.so, platform ABI, LD_LIBRARY_PATH, and the OpenCL/CUDA runtime environment

7 OTA Updates

7.1 Tool Options

Parameter	Description
-v/--version	Read the OTA tool, module, and ISP versions.
-f/--file	Specify the update file; with -a, specify the firmware directory.
-a/--auto	Automatically update FPGA and ISP firmware on both cameras.
--addr	Specify the camera I2C address; default: 0x11.
--bus	Specify the I2C bus device; default: /dev/i2c-4.
-t/--target	Specify the update target, fpga or isp; default: fpga.
-h/--helpinfo	Display help and examples.

7.2 Pre-update Checks

- Stop services using the module, I2C, CSI, or GPIO.
- Ensure a stable power supply. Do not power off or restart the system during an update.
- Record the current FPGA, ISP, and OTA tool versions. Verify the target firmware's product compatibility and checksum.
- Check the I2C bus, camera address, deserializer address, registers used to switch between the two cameras, and platform power-control GPIOs.
- Automatic updates return 0 on success and 254 on failure. Retain the complete log if an update fails.

7.3 Common Commands

```
# Query versions
sudo ./flasher_m4-g --bus /dev/i2c-4 --addr 0x11 -v

# Update the FPGA only
sudo ./flasher_m4-g --bus /dev/i2c-4 --addr 0x11 -f ./firmware.bin

# Update the ISP only
sudo ./flasher_m4-g --bus /dev/i2c-4 --addr 0x11 -t isp -
f ./ispFirmware.bin
```

7.4 Platform-specific Limitations

Automatic OTA updates switch between two cameras and control power through SoC GPIOs. The example contains platform-specific parameters. Modify and revalidate these parameters whenever the hardware revision, carrier board, connections, deserializer address, or GPIOs change.

Commands for manually switching between two cameras depend on specific deserializer registers and addresses. They must match the actual SerDes configuration; do not copy the example values unchanged. Automatic mode searches for files whose names begin with M4-g_FPGA_MRDVS and M4-g_ISP_MRDVS. Avoid multiple candidate versions in the delivery directory.

8 Troubleshooting

Symptom	What to check
Camera cannot be accessed over I2C	Check power, the GMSL control channel, deserializer links, 7-bit addressing, I2C aliases, and bus permissions.
Video nodes are missing	Check the deserializer driver, sensor sub-device, device tree endpoints, CSI controller, and kernel logs.
Video nodes exist, but STREAMON fails	Check the format, width/height, VC/Data Type, lanes and lane rate, link frequency, and Camera Framework.
No data after streaming starts	Check triggering, the 0x2000 stream-on command, whether host reception started first, GMSL lock, and CSI error counters.
Only TOF or only RGB is available	Check the corresponding VC pipe, Data Type mapping, media topology, and node bindings.
Raw TOF image looks abnormal	TOF frames contain raw algorithm input, not conventional Bayer images. Check the frame size, then pass the data to DMSDK.

Symptom	What to check
TOF and RGB are not synchronized	Check the shared Frame Sync signal, the order of blocking frame reads, frame numbers/timestamps, and buffer queue depths across the nodes.
DmOpenDevice fails	Check the I2C device name, mapped address, device type, calibration resources, permissions, and returned error code.
DmImageProcess returns -11	Check width, height, channels, type, size_per_pixel, and bytesused for TOF/RGB.
Algorithm library load failure (-25)	Check that libdmsdk.so and libdmprocess.so come from the same platform directory. Check LD_LIBRARY_PATH, the platform ABI, glibc/libstdc++, and OpenCL/CUDA runtime dependencies.
GPU backend is unavailable	For OpenCL, check the GPU driver, ICD Loader, and /etc/OpenCL/vendors. For CUDA, check the NVIDIA driver, libcuda.so.1, and related device nodes.
Memory usage grows during extended operation	Check V4L2 buffer returns through QBUF, SDK handle cleanup, logs, output buffers, and the release notes.
OTA fails or updates the wrong camera	Stop services. Check buses/addresses, camera switching, the deserializer address, GPIO power control, and firmware files.